Quality Audit Report

Executive Summary

I approached this audit as a system-quality problem, not simply as a task of grading model solutions again. The key question was: **Can we trust the results this dataset produces?**

I reviewed 21 coding tasks, their reference implementations, three model implementations per task, delivered tests, rubrics, and recorded results. The baseline audit attempted 84 evaluations and recorded 40 passing suites, 40 suites with assertion failures, and four collection failures.

The review identified issues affecting test execution, correctness, coverage, scoring, and rubric quality. Some prevent evaluation entirely; others allow incorrect implementations to pass or produce misleading scores.

My main recommendation is to establish a consistent quality gate for every task before publication. Automation should catch repeatable mechanical problems, while independent expert review should validate requirements, test oracles, and scoring fairness.

1. My Process

Establish the baseline

I first read the assessment instructions and expert guidelines, then mapped all 21 tasks to their reference implementations, Models A/B/C, delivered tests, and EvalTasks.xlsx records. I inventoried and hashed the source files to preserve the original evidence.

I wanted to distinguish three different problems: an implementation that is incorrect, an evaluation package that cannot run, and a task whose tests or scoring rules cannot reliably measure the required behavior.

Automate repeatable checks

I built a lightweight Python audit toolkit with a single PowerShell entry point. I used pytest for test execution, openpyxl for workbook inspection, and Python AST and JSON parsing for source and rubric checks.

The workflow attempted all 84 implementation evaluations in separate processes with timeouts, captured test outcomes, and checked which implementation was loaded. It also compared observed results with workbook records and checked for test-suite and rubric inconsistencies.

The baseline recorded 40 passing suites, 40 assertion-failing suites, and four collection failures. These are execution outcomes, not a measure of overall dataset quality. In particular, a collection failure means the implementation could not be evaluated; it should not be treated as a correctness score of zero.

Investigate and prioritize

Passing tests were not enough to establish task quality. I reviewed requirements, assertions, reference behavior, and rubrics, using automated checks to identify suspicious patterns for further investigation. I also replayed safely extracted prompt examples as separate experiments without changing the delivered baseline.

I prioritized issues by their impact on evaluation trust: execution blockers first, then incorrect references and expected results, misleading coverage or scores, and finally specification and design concerns.

I manually reviewed candidate findings and rejected false positives. For example, task **40133** initially appeared to have a missing-example coverage issue, but the examples were already represented in parameterized tests.

I distinguished reproduced failures from source-supported concerns and unresolved questions. The findings below reflect that evidence-based review, not simply the number of automated warnings.

2. Issues Found

Task	Issue and evidence	Why it matters
40291	Tests import test_utils.make_records, but the required helper is unavailable. The reference and all three models fail collection.	Evaluation is blocked. No reliable correctness conclusion or score can be drawn from these runs.
40278	Current and archived tests are both discovered, producing 22 collected tests instead of the workbook's expected 8. Source review also indicates a possible parser issue with a trailing newline.	Results are not comparable until the test scope and version are fixed. The parser concern needs a targeted regression test.
40228	The expected result is built using a set and converted to a list, although the prompt requires first-occurrence order. The audit reference returned ['billing', 'refund'] while the test expected the reverse.	The oracle can reject correct implementations, and set iteration can vary with hash seed. Recorded pass counts for Models A and C also differed from reproduced counts.

Task	Issue and evidence	Why it matters
40166	The reference computes pages using integer division plus one, creating an extra page when the item count is an exact multiple of page size. It passes 11/12 tests. Model C also passes 11/12 but receives 100, inconsistent with the stated proportional test allocation.	Both the reference behavior and scoring need correction. A perfect score should not be awarded without supporting evidence.
40242	Model B is recorded as passing 13/13, but reproduced 11/13. The implementation subtracts a duplicate holiday twice and returns zero for a reversed interval instead of a negative count.	Recorded results cannot be trusted until the discrepancy is explained and affected evaluations are rerun.
40174	Model B passes all ten delivered tests but raises TypeError on a missing-reading example required by the prompt. The rubric separately awards 15 points for None handling, yet the delivered tests do not exercise that behavior.	A high score can hide a required behavior failure. Coverage must reflect the contract, not just the examples.
40193	All eight tests repeat public prompt examples. Model B uses a lookup table for exact inputs, passes 8/8, and receives 100 despite the rubric prohibiting special-casing. Its fallback does not perform the required full normalization and truncation.	The task rewards example matching instead of general problem-solving, weakening its value as a benchmark.
40189	The prompt allows groups or paths in arbitrary order, but tests compare nested lists against a specific order.	Valid alternative implementations may fail because of an unstated ordering requirement.
40205	The prompt says events occur “between” endpoints without defining inclusivity, while tests require both endpoints to be included, including when start and end are equal.	The expected behavior is ambiguous. Test assumptions should not silently become requirements.

Task	Issue and evidence	Why it matters
40257	The rubric cell contains invalid JSON. Source review also suggests that an empty-string parent key is treated as no prefix, potentially collapsing distinct paths. The collision case was not reproduced in the audit.	Invalid rubric data undermines scoring reliability; the suspected reference defect needs a targeted test before it can be confirmed.
40236	Source review indicates that Model C uses an unanchored regex match that may accept a valid record followed by trailing garbage. This case was not replayed in the audit.	The tests may not fully enforce malformed-input rejection. Add a targeted test before treating the concern as a reproduced failure.
40263	The prompt permits ASCII digits and spaces/tabs only, while the reference uses \s, which accepts other whitespace. Model A uses Unicode-aware \d. These concerns were identified through source review, not targeted execution.	Implementations may accept inputs outside the contract while passing the current suite.
40305	The reference inserts email search input directly into SQL, and the rubric rewards this approach while discouraging parameterized queries.	The evaluation rewards an unsafe engineering practice rather than safe query construction.

These findings have different evidence levels: some were reproduced through execution, while others require targeted confirmation. They should be tracked separately and not treated as equally proven.

What the results say about model differentiation

The current results offer useful signals, but they do not establish benchmark quality across Grok releases.

- **40112, 40148, 40211 and 40319** show differences in model pass counts, with reference passes and recorded counts matching. They are promising candidates for further validation, not automatically approved benchmark tasks.
- **40151** has all three models passing 9/11 tests. Aggregate pass counts do not differentiate the models, although their individual failed test cases may still reveal useful differences.
- **40284** has all three models passing 9/9 tests. It is saturated in this sample and is better suited as a regression anchor than as a primary discriminator.

- **40174 and 40193** show why high pass rates can be misleading when coverage is weak.
- Tasks affected by incorrect oracles, ambiguous requirements, mixed test versions, or unreliable scoring—including **40166, 40189, 40205, 40228, 40278, 40291 and 40305**—should not be used for comparative scoring until repaired and revalidated.

A small sample of three implementations cannot establish how well a task separates future models. Differentiation should be evaluated only after the task passes quality checks.

3. Recommendations on Guidelines, Tests and Rubrics

Make task requirements testable

Define boundary behavior, ordering, invalid inputs, whitespace, character sets, exceptions, mutation, duplicates, and empty values. Map each requirement to a test or documented review check.

Test behavior, not just examples

Supplement public examples with boundary, malformed-input, duplicate-value, and combined cases.

40174 needs missing-reading tests at different positions and required failure behavior; **40193** needs unseen inputs to reject lookup tables. For **40236, 40263 and 40278**, test trailing garbage, unsupported whitespace, Unicode digits, and malformed input.

Verify defect detection through mutation testing or incorrect implementations.

Independently validate references and test oracles

Independently validate references and oracles against the contract.

- Fix the ordering logic in **40228** and test determinism across hash seeds.
- Remove the unsupported ordering constraint in **40189**, or explicitly add it to the prompt if ordering is required.
- Resolve endpoint semantics in **40205** before approving its tests.
- Verify pagination boundaries in **40166**.

Make rubrics valid, fair and evidence-based

Validate rubrics, point allocations, evidence, partial-credit rules, rounding, and score reconciliation.

Fix invalid rubric data in **40257**. Replace unsafe reference-matching criteria in **40305** with correctness, security, and maintainability criteria.

Make every release reproducible

Version task contracts, references, tests, rubrics, and dependencies. Validate collection, reference behavior, and test discovery in a clean environment.

Record test IDs, dependencies, hashes, seeds, and scoring versions. Quarantine failed tasks.

4. Recommendations for Long-Term Success

Scaling from 21 to thousands of tasks requires a quality process that is automated by default and supported by independent human review.

I would introduce a versioned task lifecycle: **draft -> independent review -> automated validation -> calibration -> release -> monitoring -> repair or retirement**. Mechanical checks should run on every task; deeper adversarial review should focus on new task families, high-risk behavior, and substantial changes.

I would organize the dataset around capabilities, not just task count. Track coverage, redundancy, difficulty, and model saturation so that new tasks address real gaps rather than repeating existing patterns.

To keep the benchmark useful as Grok improves, maintain three groups:

1. **Stable regression anchors** to detect regressions across releases.
2. **Private holdouts** to measure generalization beyond familiar examples.
3. **Frontier challenges** targeting capabilities where current models still struggle.

Keep related task variants in the same evaluation split to reduce leakage. Retire or rotate saturated tasks from headline capability measurements while retaining useful ones for regression testing. Freeze benchmark releases and rerun evaluations when task or scoring changes affect comparability.

A weekly dashboard should track release-gate compliance, blocked evaluations, reference defects, reproducibility, reviewer agreement, escaped defects, capability coverage, saturation, and evaluation cost. Report category-level results and uncertainty rather than relying only on one aggregate score.

5. Leading the Team

I would organize experts around three clear responsibilities: **author, independent reviewer, and adjudicator**. Authors build tasks, reviewers independently validate requirements and evidence, and an experienced lead resolves disagreements. Reviewers should rotate to bring fresh perspectives.

Every task would pass through contract review, construction, independent technical review, automated release checks, calibration, and post-release monitoring. No task should be published with unexplained collection failures, invalid rubric data, or unreconciled scores.

In weekly calibration sessions, the team would review real defects and disagreements. Feedback should explain what was wrong, what evidence demonstrated it, and which control would prevent recurrence. For example, **40228** should lead not only to a corrected oracle but also to a regression test and a check for the same pattern across other tasks.

I would measure:

- **Release quality:** percentage of published tasks with completed checks and independent approval; target 100%.
- **Evaluability:** collection success, reference-pass rate, and reproducibility.
- **Coverage effectiveness:** requirements covered and representative incorrect implementations rejected.
- **Reviewer calibration:** agreement on defect classification and rubric scoring before adjudication.
- **Escaped and recurring defects:** post-release defects and repeated root causes, reported with audit sample size.
- **Delivery health:** accepted tasks, first-pass acceptance, rework time, and age of blocked tasks.
- **Dataset usefulness:** capability coverage, redundancy, saturation, and trustworthy model differentiation.

I would not optimize for task volume or reviewer agreement alone. The goal is to ship useful tasks without allowing speed or shared blind spots to hide defects.

Conclusion

Trustworthy evaluations matter more than dataset size. Repair or quarantine unreliable tasks, enforce release gates, and maintain versioned evidence engineers can trust.